

Procedural trees for real-time environments

Michał Piatkowski

Computer Science
Bachelor's Thesis
15 ECTS
Spring 2026
Supervisor: Georgios Palamas
Examiner:

Procedural trees for real-time environments

Michał Piatkowski

Malmö University, Sweden

INDEX

1. Introduction	1	6.3. Games	6
1.1. Problem	2	6.3.1. Terraria	6
1.1.1. RQ	2	6.3.2. Minecraft	6
1.2. Solutions	2	6.4. Neural Networks	7
1.2.1. Forest	2	7. Future Work	7
1.2.2. Outer Space	2	7.1. Miniaturization	7
1.3. Ongoing Research	2	7.2. Better Rendering	7
1.3.1. Community	2	7.3. Persistent Homology	7
1.3.2. Industry	2	7.3.1. Mutation	7
1.4. Approach	2	7.4. Environmental Integration	7
2. Concepts	3	7.4.1. Light and Shadow	7
2.1. Design Scope	3	7.4.2. Biomes	7
2.1.1. Design Time	3	7.4.3. Placement	7
2.2. Grids	3	7.5. Scalable Noise	7
2.3. Voxels	3	7.5.1. Nodes	7
2.3.1. Concurrent Generation	3	7.5.2. Terrain Diffusion	7
2.3.2. Rendering	3	7.5.3. Boundaries	8
2.4. Level of Detail	3	8. Conclusion	8
2.4.1. Billboards	3	Glossary	9
2.5. Approximation	3	References	9
2.6. Environmental Effects	4		
3. Algorithms	4		
3.1. Growing Neural Gas	4		
3.1.1. Good Spacing	4		
3.2. Space Colonization	4		
3.3. Neural Cellular Automata	4		
3.4. Lindenmayer Systems	4		
4. Artifacts	4		
4.1. Rendering Framework	4		
4.2. Space Colonization	4		
4.2.1. Data Structures and Volume	4		
4.2.2. Speed	4		
4.2.3. Parameters	5		
4.2.4. Attractors	5		
4.2.5. Versatility	5		
4.2.6. Incremental Growth	5		
4.3. Neural Cellular Automata	5		
5. Results	5		
5.1. Space Colonization	5		
5.2. Neural Cellular Automata	6		
6. Discussion	6		
6.1. Progress	6		
6.2. Comparison	6		
6.2.1. Performance	6		
6.2.2. Customizability	6		

Abstract—Vegetation is a common part of procedurally generated environments. Level of detail techniques are used to reduce resource consumption based on distance without affecting perceivable quality. This thesis aims to explore applicable tree generation algorithms that could support level of detail for creating vast worlds in real-time. To this end, we present an optimized version of Space Colonization written in Rust.

Index Terms—generative tree modeling, models of forest trees, procedural generation, plant growth, k-nearest neighbors search, level of detail, real-time graphics

1. INTRODUCTION

Procedurally generated content has seen decent traction in games over the years. Techniques come in numerous shapes and sizes, however the *survival sandbox* genre sticks out as particularly demanding across many aspects.

The initial thesis concept went along the lines of “procedural noise-based terrain at massive scale”, stemming from the lack of real-time procedural game worlds that utilize their endless generators for extreme view distances. Improving this feature gap given limited time turned out vast, even with basis in prior art like *Minecraft (MC)*¹ or *No Man’s Sky*². This doubt lead to an early narrowing in scope towards “procedural trees”, leading us to familiar yet relevant territory.

¹<https://www.minecraft.net>

²<https://www.nomanssky.com>

Vegetation can make for effective decoration after all, often carrying the visuals by fleshing the world with familiar colors and improving immersion for players. Trees in particular play an important role due to their relative occupancy of the player’s field of view, to the extent of acting as subliminal landmarks [1]. We briefly continue this under [Section 7.3](#).

1.1. Problem

With the scope in place, we need to gaze at what it takes to bring us up to speed for real-time rendering. There is unfortunately some level of “chicken and egg” paradox at play here, where resource unavailability usually forces us into simpler “good enough” solutions, see [Section 2.1](#). Trees are common and recognizable vegetation in games, but often rely on handcrafted, reusable, and highly stylized assets. To address our gap in a manageable manner, we explore the potential of procedural generation to break visual uniformity, ideally without increasing development costs.

1.1.1. RQ:

- 1) “Which algorithms support real-time generation of trees in a performant manner?”
- 2) “How suitable are different procedural generation algorithms for real-time environment in terms of performance and controllability?”

These questions were chosen with limited time and scope in mind, while also allowing us to explore multiple techniques.

1.2. Solutions

Since the player’s viewport into the virtual world is limited, we can exploit various tricks to lighten the burden on both developers and hardware. These techniques usually fall under the term **Level of Detail (LOD)**, explained by [Section 2.4](#). The concept applies generally across game development, but is notably meaningful for trees based on their relevance in scenery, and high frequency as the predominant part of forests.

1.2.1. Forest: Realistic forests are notoriously expensive to develop and draw at high quality. At the extreme end we get workarounds like *billboards*, see [Section 2.4.1](#). For the ground itself, distant terrain can be estimated with heightmaps, but we won’t dwell on terrain in this study. Simplifying geometry is generally difficult to stylize in a dynamic environment without noticeable shadow artifacts or pop-in, so many games get by with static environments.

1.2.2. Outer Space: So far, we only described optimized rendering of full-resolution data. How about we change the data itself? Under the space-themed part of our genre, exploration games like *No Man’s Sky* and *Space Engineers*³ provide long view distance almost out of necessity. Striving for realistic scales of the universe without any sort of **LOD** is technically infeasible. These games generate data at lower resolutions from the get-go, which happen to be main driver behind this study. Both games use hierarchies of voxels to store and represent terrain, see [Section 2.3](#), albeit visualize them with *marching cubes* for a smooth look [2], compared to the cubes people may be used to when they think of voxels.

Without a planetary preview or map of nearby structures, the player would have a difficult and boring time navigating around, let alone choosing where to land their ship while not crashing into mountains at high speeds. Integration of long-distance sight with gameplay is a challenge.

1.3. Ongoing Research

Games with space exploration at their core are a minority that successfully showcase extreme view distances in a way that meaningfully integrates with gameplay. Games are generally the primary source of invention here, which means smaller representation of procedural features to borrow from.

1.3.1. Community: Around the *MC* scene, recent technical achievements regarding **LOD** are modifications (mods) to the base game, made by the community. Working with a proprietary codebase is difficult, and this is reflected in the way **LOD** is approached. Popular mods, such as *Distant Horizons*⁴ and *Voxy*⁵, generate simplified geometry from the ground truth, which can be framed as a mix of view-dependent and hierarchical **LOD**. The obvious drawback is how expensive and wasteful this process can be, generating fully detailed chunks when only a few upper chunk sections happen to be visible. We steer clear of this topic to focus on feature generation instead, pushing theories to [Section 7.5](#) on how noise could inherently be generated at different scales.

1.3.2. Industry: And for something broader, we have *Unreal Engine*⁶ technologies like *Nanite* and *Lumen*, which have been recently improved to support foliage. *Nanite* handles highly detailed geometry, and *Lumen* simulates light and shadow on top. Recent upgrades to *Lumen* allowed it to more efficiently deal with foliage by using voxelized approximations. It ties into [Section 2.5](#), but is slightly outside our paradigm. This is still an emerging area of research, since ray- and path-tracing has traditionally been, and still is, held back by hardware limitations.

1.4. Approach

Given the state of things, we can revisit an overarching problem; why do procedurally generated games struggle to present the vastness of their worlds? A simple theory is that long view distances need too large of a technical investment for an unproven feature, but a qualitative study of games would be better fit to tackle that question.

Our study tries to put technical aspects on the spotlight, but games are undeniably more than just engineering problems to be solved in a vacuum. Thus we leave many topics at just the conceptual stage, and non-technical solutions up to [Section 6](#).

Our approach in this study does not introduce any new algorithms to the table, instead acting more as an opinionated guide to which methods are suitable over others in the context of a real-time procedural game.

⁴<https://gitlab.com/distant-horizons-team/distant-horizons>

⁵<https://github.com/MCRcortex/voxy>

⁶<https://www.unrealengine.com/unreal-engine-5>

³<https://www.spaceengineersgame.com/home>

2. CONCEPTS

This section means to give background on some concepts. It does not intend to be a definitive or complete explanation.

2.1. Design Scope

When crafting worlds for games, the designer tends to distribute resources and skills in a way that ends up noticeable to the player. This tendency has a decisive effect in the context of visual features; nearby objects get more attention from the player and in turn the designer, while distant objects lose importance. In linear or closed-world games, the designer can deliberately choose where to allocate resources since playable area is limited.

2.1.1. Design Time: Open-world games, especially those relying on procedural generation, greatly complicate resource allocation. We can no longer focus on discrete parts but have to instead produce a coherent whole. To cope with the vast search space, the designer may intentionally impose rules and constraints, forming a *design language* with basis in available resources, target audience, and the artistic vision.

A design language helps from a technical standpoint too. Rules can guide the designer towards handcrafted and fitting content, over spending resources on a powerful procedural system which carries the risk of not fitting in aesthetically. Authored content can also be easier to optimize for runtime.

2.2. Grids

Partitioning space into grids can be implicit like rooms in a dungeon, or explicit like individual voxels in terrain. Herein lies the tradeoff; grid size becomes one of the deciding factors on how the designer spends their resources.

Take some of the possible features that can appear in an infinite sandbox game like terrain, biomes, vegetation, or structures – listed roughly from large to small. These need to be mixed in some way to make an engaging experience, but in a procedural world, we do not know where the player will venture or the order in which they encounter features. We can guide them with specific cues, but it can be challenging to generate meaningful trails, such as desire paths stretching across settlements, or landmarks that signal the presence of some nearby point of interest.

Hardware limitations are a driving force as well, where grids can be particularly versatile in accommodating memory limits, but also introduce parallelization opportunities.

2.3. Voxels

Partitioning space can help us with *separation of concerns* in the distributed system that is our game world. To better explain how one can utilize a hierarchy of grids, we can use MC as an example:

- 1) *Blocks* are the smallest unit i.e. the voxels.
- 2) *Sections* are portions containing 16^3 blocks.
- 3) *Chunks* are columns of sections – height can vary.
- 4) *Regions* are groups of 32×32 chunks.

Anything beyond blocks is practically just technical details for a player, but to us it can explain some technical intricacies.

2.3.1. Concurrent Generation: The world is generated chunk-wise and over multiple consecutive steps, all on demand as chunks appear in the player's view distance [3]. Structure origins and biomes are calculated before blocks can be placed. Terrain shaping follows, and is a computationally expensive phase thanks to various density functions. The convenient property of these formulas is their continuity, given that most of them are multiple octaves of gradient noise with some linear arithmetic sprinkled in between. Density calculations are deterministic and embarrassingly parallel, in other words.

Caves and cliffs are fun and all, but there is a concurrency problem soon after; the decoration post-processing steps may place structures, vegetation, and ore – to name a few. The point of congestion is the possibility of decorations intersecting multiple chunks, which is solved by tracking such dependencies and finalizing later in a pre-defined order.

2.3.2. Rendering: Chunks render only when fully generated. This is normal for chunking since it is easier cache and invalidate, and avoids the substantial overhead of maintaining a rapidly shifting preview for incomplete data.

2.4. Level of Detail

At the beginning of 3D graphics, avoiding unnecessary work was crucial to render scenes in real-time. Rasterization was the bottleneck, so painting only visible parts of polygons front-to-back could skip a lot of processing [4]. The bottleneck has since shifted wildly, orienting the solution space around software design rather than hardware limitations, yet the spirit of avoiding work lives on.

This leads us to a perhaps literal interpretation of approximation – LOD – referring to the complexity of a 3D model, commonly recognized by the act of reducing detail based on distance [5]. Triangle meshes can be simplified by removing vertices that contribute little based on perspective, most often baked algorithmically into a couple variants as part of an offline content pipeline.

2.4.1. Billboards: At the extreme end, 3D models can be pre-rendered into a 2D sprite, known as a *billboard*, usually with a normal map to help lighting.

2.5. Approximation

This premise of estimates applies across disciplines and is an essential tactic for simulating complex natural phenomena on constrained digital hardware.

Borrowing ideas from nature is referred to as *biomimicry*, and may be one of the most self-evident approaches in replicating natural structures and behaviors [6], [7]. The important takeaway is, we do not need to simulate life in microscopic detail to recreate useful mechanisms, as will become evident through Section 3.2.

This also ties us back to Section 2.1. For games we are usually encouraged to bend the rules, especially if the artistic vision is not in need of realism, or when technical limitations pressure us into finding more performant solutions. All this while subjects need to fit in a given media without feeling out of place.

2.6. Environmental Effects

By exploring biomimicry, we may uncover how the surrounding environment can guide and drive a realistic simulation. Finite resources are a natural constraint that helps create self-organizing systems. That makes implementing resources in a simulation quite intuitive.

When searching outside the real-time domain, there exists plentiful research for beautiful and realistic tree models that get far without spinning up complex recreations of the physical world, see [Section 7.4](#) for examples.

3. ALGORITHMS

Methods listed in this section are inspired by the ideology outlined in [Section 2.5](#), most of them to high degree. The goal is to find something suitable for real-time generation, ideally supporting LOD, all while balancing usability for designers.

3.1. Growing Neural Gas

Our supervisor kickstarted the exploration by suggesting Growing Neural Gas (GNG) networks [8] when the initial goal was being established. Even after exploring various implementations, we concluded that GNG was not particularly relevant for growing trees, nor was it effective for real-time even with major optimizations [9].

3.1.1. Good Spacing: Possible uses we came up with were related to creating bounding geometry for tree crowns by training it on natural shapes and existing trees. Constructing meshes for rendering was also an idea, but grew out of scope.

On the subject of GNG, we also got to explore the LOD aspect and found interesting applications for compressing point cloud colors [10], but without advance in growing trees.

3.2. Space Colonization

In search of faster solutions, Space Colonization (SC) was the most promising algorithm we encountered [11]. Thus it became the main subject and was examined under [Section 4.2](#).

SC is lenient when it comes to choice of data structures, which led to interesting findings, see [Section 4.2.2](#). Reacting to the environment also comes naturally, see [Section 7.4](#).

3.3. Neural Cellular Automata

When growing this approach may look adjacent to image diffusion, but integration of cellular automata sets them apart.

In spirit of *Conway's Game of Life*, but with a complex black-box for rules, we get the aptly named Neural Cellular Automata (NCA), which we also explore under [Section 4.3](#).

NCA generates structures through repeated local interactions between cells [12]. Instead of teaching large Neural Networks (NNs) to refine noise in a top-down strategy, each cell maintains a state and uses a small NN to determine how that state should change based on neighbouring cells. Starting from only a few seed cells, these local updates can gradually produce complex global structures. This makes NCA particularly interesting for procedural vegetation, since plants similarly develop through local growth processes and can potentially respond to environmental conditions. However, the limited controllability and computational requirements of

NCA may make it less suitable for real-time tree generation than more explicitly controlled approaches such as SC.

3.4. Lindenmayer Systems

Also known as L-systems, they are a formal grammar and rewriting system, with good examples in [13].

These can be grown over multiple steps like SC, but may require significantly more effort to miniaturize due to the recursive nature of the algorithm. Adding LOD essentially requires dedicated production rules that cause splits into defined levels, making authorship difficult.

4. ARTIFACTS

4.1. Rendering Framework

To ease experiments regarding our algorithms of choice, we developed a minimal framework with the primary purpose of visualizing point clouds, since most of the data we are working with is effectively represented by points.

The choice of tech stack was driven by the performance-oriented nature of this research, which is why we settled on Rust early and out of familiarity. This gave us access to high-quality libraries that provide essentials like graphics (wgpu), immediate-mode interface (egui), and acceleration structures (rstar).

4.2. Space Colonization

We implemented the SC algorithm by following instructions in the tree modeling paper [11]. Nearest Neighbor Search (NNS) is a fundamental part of the algorithm, and a $O(n^2)$ loop over all attractors is untenable.

4.2.1. Data Structures and Volume: Thus an octree library (oktree) was used as the first acceleration structure. Octrees served us well throughout development, but it became apparent that iterating over many large overlapping spheres was expensive when influence radii d_i on attractors were large. Naively constraining the search to a certain radius would deviate us from the algorithm, in turn worsening growth dynamics by disregarding distant attractors.

Realising that we worked with volumes and not just points, we pursued a spatial hash with cells that each referenced all intersecting attractors. These amortized lookups gave a decent speedup, but increased both memory usage and removal time due to excessive duplication. Cost remained high even while utilizing packed references as small as 16-bit; enough for 65535 attractors, or 2^{16} indices minus 1 tombstone. Generally referred to as a *memory arena*, it allows self-referential indices and can simplify deallocation by dropping whole ranges.

The structures we explored usually trade construction time and memory for improved lookup times. This was the reason for us looking beyond Voronoi diagrams in the first place, specifically with expensive Delaunay triangulation needed to construct them. An aspect we paid less attention to, but is worth noting, is tree balance and quality, which can vary at runtime as attractors are killed and nodes are spawned.

4.2.2. Speed: So far, neither octrees or spatial hashing proved effective against slowdown caused by smaller segment size D . Our supervisor seemed to recognize this problem as

related to *DBSCAN* [14], leading to our final structure of choice; the *R*-tree* [15]. Using an existing library (*rstar*), we managed to further decrease lookup time and trivialize construction cost while creating an optimal tree since we have all points upfront, all while maintaining fast *NNS* regardless of influence radius. This was now a faithful reimplementation of the *SC* algorithm, with informal metrics in Table 1.

4.2.3. Parameters: The main variables d_i , d_k , and D can lead to very different generation times for a particular attractor cloud. Smaller segments lead to fewer attractors potentially being killed per new node, increasing the amount of iterations needed to consume all reachable attractors. Our choice of data structures may greatly alleviate the cost of large influence distances, but this variability can fundamentally not be eliminated without a different growth strategy and would likely require a new approach overall.

The main variables are far away from a “polished product” like presented in section 4.1 of [16]. Even those can be further expanded upon through e.g. alternating node activations or non-immediate attractor death. We did not try anything fancy since this relies too much on game context.

4.2.4. Attractors: Parameters are only half the story, with the placement of attractors giving a tree some distinct form. The distance between and grouping of points determines where branches grow toward and consequently split. Nodes do not actually “branch out” though – previous nodes get assigned outstanding attractors as big groups are consumed. Treating nodes as buds is also a clever trick, see Section 7.4.1.

Future work from *SC* paper(s) mention novel ideas for generating attractors. One fun idea is to convert 3D scans of real trees to point clouds, though the parameters are not easy to just guess from scans. On a more technical note, we prefer sampling *Signed Distance Field* (SDF) functions to spawn attractors, which would be performant and easy for designers to pick, combine, and visualize.

4.2.5. Versatility: With our real-time requirement, we can worry less about theoretical time complexities. We can tune both parameters and data, allowing designers to also allocate time towards post-processing of the generated skeleton. Point clouds are unlikely to be useful in their raw form, but the methods for sprucing them up are vast [16].

The *SC* paper already mentions decimation and subdivision along branch curves, but these operate on points to improve detail. An example that is more relevant to us, and closer to the tail end of the content pipeline, is voxelization; the act of turning smooth geometry into a rigid grid. A simple ray-cast that sets a block per step can be used to simulate subdivision.

4.2.6. Incremental Growth: *SC* can be grown over multiple steps i.e. over multiple game frames or ticks, allowing for massive structures to appear without a hitch. There is also the quirk of the tree growing in a natural bottom-up direction, and if desired, a growth animation can be used instead of pop-in.

4.3. Neural Cellular Automata

This was pursued since a few of the pre-trained examples were of trees [17], giving a useful baseline that worked. We reproduced the open-source build on GitHub⁷, which included updating some yanked Python packages. This allowed us to explore the provided *Jupyter notebooks*, and train *NCA* from scratch using the *MC* server controlled by Python.

The original loss function is a combination of *Softmax* and *negative log likelihood* loss⁸. We attempted to modify the loss function to achieve different growth patterns. That may be effective for tree variation but was difficult to control. The growth is too sporadic and unnatural, and generally struggles to settle on novel shapes outside of training data. We found a paper that takes a different approach to automata using convolutional *NNs* [18], and the method presented there seemed more plausible than messing with *NCA* internals.

5. RESULTS

Hardware:

- CPU: AMD Ryzen 7 7700X 8-Core
- RAM: 2 × 32 GB, DDR5 5200 MT/s, CL32

5.1. Space Colonization

Our *SC* implementation as described in Section 4.2. The figures below showcase a simple render of input attractor clouds and output trees. The box is filled by 20000 points, randomly distributed across $200 \times 225 \times 200$ units. The initial node is spawned 75 units below the center.

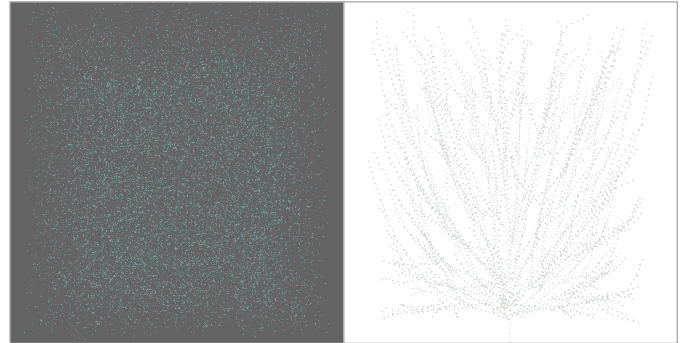


Figure 1. *SC* box before growth.

Figure 2. *SC* box after growth.

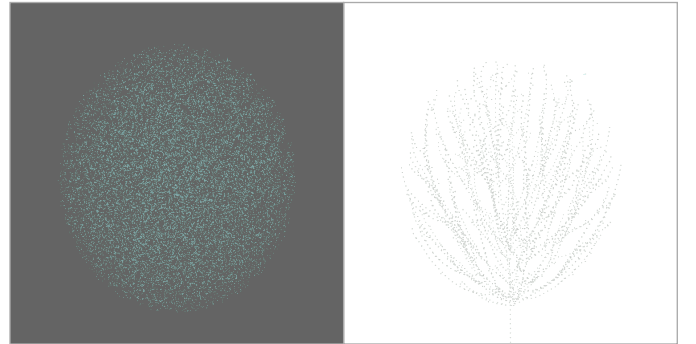


Figure 3. *SC* egg before growth.

Figure 4. *SC* egg after growth.

⁷<https://github.com/real-itu/3d-artefacts-nca>

⁸<https://docs.pytorch.org/docs/stable/nn>

	Construct	Lookup	Memory
Octree	100ms	1500ms	High
Spatial Hash	200ms	800ms	Medium
R*-tree	1.5ms	300ms	Low

Table 1. Data structure metrics for 20000 randomly distributed attractors.

5.2. Neural Cellular Automata

The generation of Fig. 5 converges after 80 iterations, or roughly 34.7 seconds on CPU. On the other hand, introducing randomness to cell propagation creates spurious growths seen in Fig. 6. Growth quickly collapses when scaling weights with values less than one, making the tree disappear.

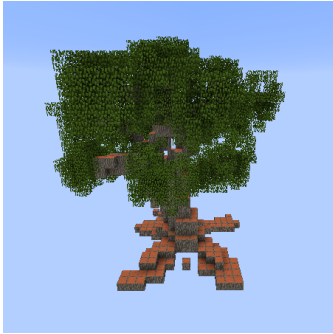


Figure 5. Original NCA tree

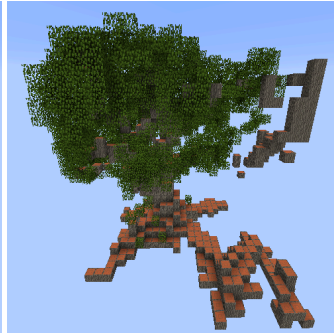


Figure 6. NCA tree with factors: mask = 0.2, fire = 0.75

6. DISCUSSION

The algorithms we looked at involved a lot of sense-making, especially when shoe-horning tree generation into them [7]. This is apparently a hot topic for modern systems and black-boxes, and rightfully so. Emergent behavior from small cells that interact is what nature is all about, and this research was a pleasant reminder of that. Getting steered into a stochastic process was enlightening, as someone often too focused on the reductionism that occurs in engineering.

6.1. Progress

We started with broad search for any research related to growing shapes. Then we evaluated a select few (GNG, SC, NCA) to assess strengths. Candidates that passed the threshold moved on to the coding playground. The goal of generating trees was most important, but NCA made us think about other vegetation and even structures. By exploring prior work, we got a grasp on the possibilities.

Many experiments had to be cut short as they would need their own dedicated research, see Section 7. The feasibility of modifying algorithms for e.g. LOD remains unknown.

6.2. Comparison

6.2.1. Performance:

6.2.1.1. SC: It is difficult to make a performance comparison with the original Voronoi diagram solution in [11] because they do not list concrete numbers, and we were not able to reproduce their setup. Suffice to say, constructing a R*-tree in bulk should be in the same ballpark as building a

Voronoi diagram, as both structures self-balance in a similar manner. We did find an optimized KD-tree construction [19], but the numbers in their figures 4 and 5 do not compare against R*-tree, only R-tree, so we decided to not test KD-tree.

This was a pain-point we solved from the future-work section of [11]. A benefit of the R*-tree is that segment size D does not significantly affect lookup performance. Insertion parameters can be tuned, but only for marginal gains.

6.2.1.2. NCA and GNG: NCA, as presented with voxels, is not suitable for real-time generation. GNG was not tested since it was not used. Training is very slow for NN techniques in general, and inference is inefficient on CPU:s.

6.2.2. Customizability:

6.2.2.1. GNG: Needs to be trained on an existing model. The more you train, the better the point-cloud approximation of the model. We did not see how this could be used for tree generation directly, but findings in Section 3.1.1 could be applied to spawn attractors (Section 4.2.4) at more optimal distances, since GNG clouds can be pre-computed at design-time and evaluated cheaply later.

6.2.2.2. NCA: Needs to be trained on the cells i.e. pixels or voxels of an existing shape. After training, it will attempt to reproduce the shape at all costs. Modifying the internals, like liveness mask or weights, is not effective in customizing the shape, and it is trivial to make the shape explode.

6.2.2.3. SC: No training required. The designer has high control over the tree shape through various parameters and attractor cloud, and it is very feasible for developers to add new behaviors and node interactions.

Our artifact pales in comparison with mentioned papers in all aspects except performance. There was not enough time to recreate all the knobs or visuals, but we made sure the R*-tree would not prohibit future expansions.

The authors of [11] were proud that their solution “grows like actual trees”, and we agree with the sentiment. This also ties well into Section 2.5 and Section 2.6.

6.3. Games

Most of the inspiration behind this research came from the games we enjoy, with two of the primary ones listed below.

6.3.1. Terraria: Living Trees in Terraria⁹ are structures that spawn on the surface and in underground jungles. Players enjoy building bases around or inside such trees, to the point of the developers increasing the likelihood of trees spawning in groups, for players to build overhang treehouses¹⁰.

6.3.2. Minecraft: Vanilla MC is not very creative when it comes to big trees, the best we have are tall jungle and spruce trees. There are plenty of player builds of treehouses and such out there, but we want to bring up a fun mod instead.

The original mod is not maintained anymore, but a stand-alone is available¹¹, where trees can span multiple regions of chunks. Mods are also convenient because we can look at their code and see how large trees or structures are generated in practice, giving us a glimpse into upsides and pitfalls.

⁹<https://www.terraria.org/>

¹⁰https://terraria.wiki.gg/wiki/Living_Tree#History

¹¹<https://github.com/Blue-Beaker/Sacred-Trees>

6.4. Neural Networks

Machine learning was a recurring topic during our search, and the pool of papers was difficult to filter from the frequent occurrence. Perhaps unsurprising was the interplay between point clouds of terrain and vision systems for self-driving cars, but we digress. Since our priorities center around real-time generation, we tried to look away from prominent solutions such as transformers or diffusion. These techniques of today, even considering modern hardware, leave a lot to be desired across efficacy, efficiency, controllability, performance, and training. This did not stop others in the field from attempting to replace noise-based terrain with something more aesthetic and novel, see [Section 7.5.2](#).

7. FUTURE WORK

7.1. Miniaturization

Let us address the elephant in the room first; our artifact does not measure or represent any [LOD](#) features. The initial premise was to investigate the viability of these algorithms in a real-time environment, with the crux being that it should be scalable for large procedural worlds. We tried to lay the groundwork for such research instead, and believe [SC](#) to be the best fit, at least algorithmically.

[NCA](#) papers mention the possibility of scaling beyond individual cells, but we could not deduce how effective that would be in practice, and whether it could apply to vegetation or only subdivisible city-like structures.

7.2. Better Rendering

Our minimal rendering framework was mostly used to just visualize data. The trees it can draw are not at all interesting compared to prior work, but for the type of work we did, the iteration speed was worth the setup. The useful things missing were text rendering and gizmos, which would help debug node numbers in 3D space.

Existing game engines should have no problem at achieving the visual quality of [\[20\]](#). Textured materials in our artifact would be nice to have.

7.3. Persistent Homology

Persistent Homology is “designed to uncover geometric and topological features of data across multiple scales” [\[21\]](#). In other words, it could be used to measure and compare trees through “barcodes”. Turning subjective geometry into objective descriptions is a complex topic, but this bridge between qualitative and quantitative studies is very useful.

7.3.1. Mutation: For a biologically grounded comparison, let us reimagine the barcodes as mutations and genes.

An obvious gene could then be coding for tree height, made easier if accounting for perspective or distance to the player. This correlates with the subliminal effect per the introduction.

We also explored a collection of projects where the goal is mutating 2D patches to resemble famous paintings¹². It was refreshing to see abstract triangles freely shifting after the adventure with [NCA](#).

7.4. Environmental Integration

7.4.1. Light and Shadow: Shadow propagation, as introduced by [\[20\]](#), was a great nature-inspired method we liked. It augments [SC](#) by constraining branch splits to buds, and by propagating a penumbra from buds, “consuming light” from buds below. One could say that attractors in [SC](#) are themselves a form of limited resource, and this optimized shadow-caster took it a step further. From a glance we expect this to be even faster than [SC](#) on its own, even with the added functionality, and we would like to compare them in the future.

[Section 2.6](#) was so hot at the start that we naively wasted time trying to combine shadow propagation with [NCA](#) in the [MC](#) environment. [NCA](#) was too difficult to stabilize with dynamic conditions disturbing it, so we moved back to [SC](#).

7.4.2. Biomes: Our research focused on one tree in a vacuum without a direct plan forward, instead leaving room for future integrations with neighboring vegetation and terrain. There are *many* variables that can affect plant growth. Our favorite that came from a brainstorm session are localized species such as insects and mycelium, which may act as micro-biomes that dictate tree attributes. All these systems could interact with each other and create interesting ecospheres.

The weather is also an important world-building tool [\[22\]](#) that extends both biome and placement aspects.

7.4.3. Placement: We can not propose a definite optimal way to place trees, since placement in a scene can depend on endless factors. We recommend to keep surrounding world and [Section 2.5](#) in mind. Try integrating with the terrain generator to avoid expensive search-based rules. At least there is the benefit of this being mostly one-directional; trees are shaped by terrain, and do not usually shape terrain themselves.

7.5. Scalable Noise

The initial concept and title for this research project was “Generating noise at different [LOD](#)”. This can be implemented with textured heightmaps, as is done in *Veloren*¹³. We are thinking of taking that further by generating noise at reduced [LOD](#) directly, skipping expensive downscaling.

7.5.1. Nodes: We should investigate how much effort it would be to design a node-based terrain generation system, akin to [MC](#) density functions usable from datapacks, from the ground up while incorporating [LOD](#) features from the very beginning. Existing systems such as [MC](#) are difficult to rewrite in a way that is backwards-compatible in a [LOD](#) paradigm. The [MC](#) systems were simply not built for scale, which really showed when the world height was increased in the “Caves and Cliffs” update and the game had to raise minimum hardware requirements. A freshly designed system should be written with parallelism and determinism in mind, to avoid the painful pitfalls of [MC](#).

After many years in development, *Distant Horizons* recently gained support for generating a rough heightmap of “the entire world”, which transitions us nicely to the next section...

7.5.2. Terrain Diffusion: Picking up from [Section 6.4](#) and above, we want to mention *Terrain Diffusion*¹⁴ for [MC](#) because

¹²<https://github.com/peterbraden/genetic-lisa>

¹³<https://veloren.net/>

¹⁴<https://github.com/xandergos/terrain-diffusion-mc>

it is quite the achievement. The backbone is the *InfiniteDiffusion* framework developed for this purpose, permitting cheap manipulation of unbounded tensors.

7.5.3. *Boundaries*: It is somewhat unclear how to handle features and structures under chunked grids in a good way, which is relevant even for small trees. Anything that can cross chunk boundaries usually needs special care. If we take *Hytale*¹⁵ as a freshly built example, the engine attempts to hide most chunk-boundary details away from the designer. This is convenient, but can prohibit design of larger structures.

8. CONCLUSION

In attempt to answer the research question(s): **SC** has high potential for real-time generation of large and realistic trees at various **LOD**, with plenty of controllability all around.

Most techniques mentioned in this paper need to be part of a bigger system to give proper contrast to up-sides and down-sides. These algorithms are rarely usable out-of-the-box, especially when they need to be adapted for custom engines.

With the advent of diffusion models and other such techniques, it remains to be seen whether classical solutions like noise functions and L-systems will stand their ground in procedural generation against *InfiniteDiffusion*.

Data structures get you far, some just get you farther!

¹⁵<https://hytale.com/news/2026/1/the-future-of-world-generation>

GLOSSARY

GNG	Growing Neural Gas
LOD	Level of Detail
MC	Minecraft
NCA	Neural Cellular Automata
NN	Neural Network
NNS	Nearest Neighbor Search
SC	Space Colonization
SDF	Signed Distance Field

INDEX OF FIGURES

Fig. 1 SC box before growth.	5
Fig. 2 SC box after growth.	5
Fig. 3 SC egg before growth.	5
Fig. 4 SC egg after growth.	5
Fig. 5 Original NCA tree	6
Fig. 6 NCA tree with factors: mask = 0.2, fire = 0.75 ...	6

INDEX OF TABLES

Table 1 Data structure metrics for 20000 randomly distributed attractors.	6
--	---

REFERENCES

- [1] P. H. Jęger, N. C. Nilsson, and G. Palamas, “Can’t see the Forest for the Trees: Perceiving Realism of Procedural Generated Trees in First-Person Games,” *EAI Endorsed Transactions on Creative Technologies*, vol. 8, no. 26, Mar. 2021, doi: [10.4108/eai.16-3-2021.169029](https://doi.org/10.4108/eai.16-3-2021.169029).
- [2] W. E. Lorensen and H. E. Cline, “Marching cubes: A high resolution 3D surface construction algorithm,” *SIGGRAPH Comput. Graph.*, vol. 21, no. 4, pp. 163–169, Aug. 1987, doi: [10.1145/37402.37422](https://doi.org/10.1145/37402.37422).
- [3] “Minecraft Wiki.” Accessed: May 20, 2026. [Online]. Available: <https://minecraft.wiki/>
- [4] D. Gordon and S. Chen, “Front-to-back display of BSP trees,” *IEEE Computer Graphics and Applications*, vol. 11, no. 5, pp. 79–85, 1991, doi: [10.1109/38.90569](https://doi.org/10.1109/38.90569).
- [5] T. K. Heok and D. Daman, “A review on level of detail,” in *Proceedings. International Conference on Computer Graphics, Imaging and Visualization, 2004. CGIV 2004.*, 2004, pp. 70–75. doi: [10.1109/CGIV.2004.1323963](https://doi.org/10.1109/CGIV.2004.1323963).
- [6] J. F. Vincent, O. A. Bogatyreva, N. R. Bogatyrev, A. Bowyer, and A.-K. Pahl, “Biomimetics: its practice and theory,” *Journal of The Royal Society Interface*, vol. 3, no. 9, pp. 471–482, Apr. 2006, doi: [10.1098/rsif.2006.0127](https://doi.org/10.1098/rsif.2006.0127).
- [7] K. E. Weick, K. M. Sutcliffe, and D. Obstfeld, “Organizing and the Process of Sensemaking,” *Organization Science*, vol. 16, no. 4, pp. 409–421, Jul. 2005, doi: [10.1287/orsc.1050.0133](https://doi.org/10.1287/orsc.1050.0133).
- [8] B. Fritzke, “A growing neural gas network learns topologies,” *Advances in neural information processing systems*, vol. 7, 1994.
- [9] D. Fišer, J. Faigl, and M. Kulich, “Growing neural gas efficiently,” *Neurocomputing*, vol. 104, pp. 72–82, 2013, doi: [10.1016/j.neucom.2012.10.004](https://doi.org/10.1016/j.neucom.2012.10.004).
- [10] S. Orts-Escolano, J. Garcia-Rodriguez, V. Moreli, M. Cazorla, and J. M. Garcia-Chamizo, “3D colour object reconstruction based on Growing Neural Gas,” in *2014 International Joint Conference on Neural Networks (IJCNN)*, Jul. 2014, pp. 1474–1481. doi: [10.1109/IJCNN.2014.6889546](https://doi.org/10.1109/IJCNN.2014.6889546).
- [11] A. Runions, B. Lane, and P. Prusinkiewicz, “Modeling trees with a space colonization algorithm,” in *Proceedings of the Third Eurographics Conference on Natural Phenomena*, in NPH’07. Prague, Czech Republic: Eurographics Association, 2007, pp. 63–70. doi: [10.5555/2381384.2381395](https://doi.org/10.5555/2381384.2381395).
- [12] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, “Growing Neural Cellular Automata,” *Distill*, 2020, doi: [10.23915/distill.00023](https://doi.org/10.23915/distill.00023).
- [13] O. Blomqvist, P. Kraft, H. Lidin, R. Motzheim, A. Tonderski, and G. Wagner, “Generating Compelling Procedural 3D Environments and Landscapes,” 2016. [Online]. Available: <https://hdl.handle.net/20.500.12380/244588>
- [14] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, in KDD’96. Portland, Oregon: AAAI Press, 1996, pp. 226–231. doi: [10.5555/3001460.3001507](https://doi.org/10.5555/3001460.3001507).
- [15] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, “The R*-tree: an efficient and robust access method for points and rectangles,” in *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data*, in SIGMOD ’90. Atlantic City, New Jersey, USA: Association for Computing Machinery, 1990, pp. 322–331. doi: [10.1145/93597.98741](https://doi.org/10.1145/93597.98741).
- [16] R. Hoetzlein, “A Procedural Model for Diverse Tree Species,” in *Proceedings of the 17th International Conference on the Foundations of Digital Games*, in FDG ’22. Athens, Greece: Association for Computing Machinery, 2022. doi: [10.1145/3555858.3564251](https://doi.org/10.1145/3555858.3564251).
- [17] S. Sudhakaran *et al.*, “Growing 3D Artefacts and Functional Machines with Neural Cellular Automata,” 2021. doi: [10.1162/isal_a_00451](https://doi.org/10.1162/isal_a_00451).
- [18] D. Zhang, C. Choi, J. Kim, and Y. M. Kim, “Learning to Generate 3D Shapes with Generative Cellular Automata,” 2021. doi: [10.48550/arXiv.2103.04130](https://doi.org/10.48550/arXiv.2103.04130).
- [19] Y. Narasimhulu, V. Venkaiah, R. Pasunuri, and A. Suthar, “CKD-Tree: An Improved KD-Tree Construction Algorithm,” in *ISIC 2021: International Semantic Intelligence Conference*, New Delhi, India, Feb. 2021, pp. 211–218. [Online]. Available: <https://ceur-ws.org/Vol-2786/Paper28.pdf>
- [20] W. Palubicki *et al.*, “Self-organizing tree models for image synthesis,” in *ACM SIGGRAPH 2009 Papers*, in SIGGRAPH ’09. New Orleans, Louisiana: Association for Computing Machinery, 2009. doi: [10.1145/1576246.1531364](https://doi.org/10.1145/1576246.1531364).
- [21] A. J. Kemme and C. A. Agyingi, “Persistent Homology: A Pedagogical Introduction with Biological Applications,” 2025. doi: [10.48550/arXiv.2505.06583](https://doi.org/10.48550/arXiv.2505.06583).
- [22] S. Pirk, T. Niese, T. Hädrich, B. Benes, and O. Deussen, “Windy trees: computing stress response for developmental tree models,” *ACM Trans. Graph.*, vol. 33, no. 6, Nov. 2014, doi: [10.1145/2661229.2661252](https://doi.org/10.1145/2661229.2661252).